

3.1.2 Normalization

10 June 2025 19:15

◆ What is Normalization?

- **Definition:**
Normalization is a data preprocessing technique used in machine learning to adjust the values of numerical columns in a dataset to a common scale *without distorting differences in the ranges of values*.
- **Purpose:**
It helps in bringing different units or scales (like weight in kg vs height in cm) to a uniform range, making algorithms more efficient and comparable.

💡 Why is Normalization Important?

- Many machine learning algorithms work better when features are on the same scale.
- Features with larger ranges might dominate others and skew results.
- Especially important when using distance-based algorithms (like KNN, SVM).

🏠 Real-Life Example

- Suppose you are predicting whether a person will buy sports equipment based on features like **weight**.
- Weight could be in **grams, kilograms, or pounds** – leading to inconsistent scaling.
- To fix this, all values need to be **converted and scaled uniformly** before feeding them into a model.

🧰 Types of Normalization Techniques

1. **Min-Max Scaling (Most Common)**
 2. **Mean Normalization**
 3. **Max Abs Scaling**
 4. **Robust Scaling**
- 💡 90% of the time, **Min-Max Scaling** is used in practical ML scenarios.

🔧 Min-Max Scaling Explained

✅ **Formula:**

$$X_{\text{scaled}} = \frac{X - X_{\min}}{X_{\max} - X_{\min}}$$

- X : Original value
- $X_{\min}$: Minimum value in the column
- $X_{\max}$: Maximum value in the column
- X_{scaled} : New normalized value

✅ **Effect:**

- Converts all values into the range **[0, 1]**
- Ensures the minimum value becomes **0** and the maximum becomes **1**

🔍 Example Calculation

Given a feature column:

sharp

📋 Copy 🗑️ Edit

[130, 60, 60, 32, 50]

- $X_{\min} = 32$
- $X_{\max} = 130$

📊 Properties of Min-Max Scaling

- Resulting values always lie between **0 and 1**
- Doesn't change the **distribution shape**, only rescales it
- Sensitive to **outliers** (because it uses min and max)

🌱 Geometric Intuition

- Imagine you have 2 features: **Weight (W)** and **Height (H)**
- Plotting them on a graph results in some scattered points.
- After min-max scaling:
 - The data points are **compressed into a unit square** (0 to 1 along both axes)
 - In higher dimensions, this becomes a **unit cube, unit hypercube**, etc.
 - You are essentially "squeezing" your dataset into a fixed bounded space.

Conclusion

- Normalization is essential when dealing with numerical data of varying units or scales.
- Among all, **Min-Max Scaling** is the most frequently used technique due to its simplicity and effectiveness.
- Helps improve performance and convergence speed of many machine learning algorithms.

Wine Dataset Normalization & Visualization

Dataset Overview:

```
[1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

[2]: df = pd.read_csv('wine_data.csv', usecols=[0,1,2])
df.columns = ['class_label', 'alcohol', 'Malic Acid']
df.head()

[3]:
```

	class_label	alcohol	Malic Acid
0	1	13.20	1.78
1	1	13.16	2.36
2	1	14.37	1.95
3	1	13.24	2.59
4	1	14.20	1.76

- A wine dataset was used, containing features like:
 - Alcohol
 - Malic Acid
 - Class Label (Target variable: 3 classes - 1, 2, and 3)
- Alcohol had a larger magnitude compared to Malic Acid.

Initial Distribution Analysis:

- Used Seaborn's **distplot** to visualize:
 - Alcohol had a near-normal distribution.
 - Malic Acid had a more skewed distribution.

Data Preparation:

1. Extracted and loaded the dataset using Pandas.
2. Selected the three relevant columns: Alcohol, Malic Acid, Class Label.

```
[9]: from sklearn.model_selection import train_test_split as tts

x_train,x_test,y_train,y_test = tts(df.drop('class_label',axis = 1),
                                   df['class_label'],
                                   test_size = 0.3,
                                   random_state = 0)

x_train.shape,x_test.shape
```

```
[9]: ((123, 2), (54, 2))
```

```
[26]: from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
scaler.fit(x_train)
x_train_scaled = scaler.transform(x_train)
x_test_scaled = scaler.transform(x_test)
```

Normalization:

- Applied **Min-Max Normalization**:
 - Scales data in the range [0, 1].
 - Formula:

$$X_{\text{normalized}} = \frac{X - X_{\min}}{X_{\max} - X_{\min}}$$

- After normalization:
 - Min value became 0
 - Max value became 1
 - Verified via `.describe()` method.

```
[40]: x_train_scaled = pd.DataFrame(x_train_scaled, columns = x_train.columns)
x_test_scaled = pd.DataFrame(x_test_scaled, columns = x_test.columns)
x_train_scaled.describe()
```

```
[40]:
```

	alcohol	Malic Acid
count	123.000000	123.000000
mean	0.523079	0.299676
std	0.209894	0.237984
min	0.000000	0.000000
25%	0.360215	0.136456
50%	0.543011	0.191446
75%	0.697581	0.445010
max	1.000000	1.000000

Transformation Process:

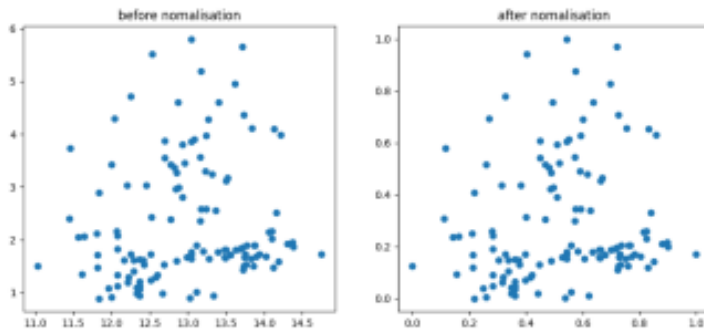
- Used MinMaxScaler from sklearn.preprocessing.
- Applied .fit_transform() on training and test datasets separately.
- Converted normalized NumPy arrays back to DataFrames for compatibility.

Post-Normalization Analysis:

- Replotted scatter plots and distribution plots:

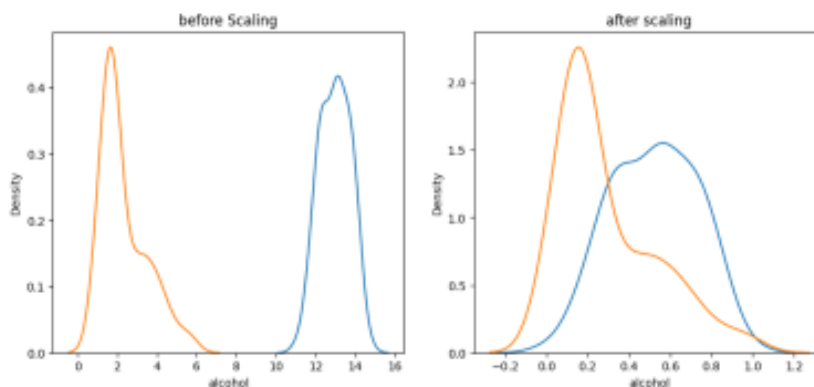
```
[38]: fig,(ax1,ax2) = plt.subplots(ncols = 2, figsize=(12, 5))
      ax1.scatter(x_train['alcohol'],x_train['Malic Acid'])
      ax1.set_title('before normalisation')
      ax2.scatter(x_train_scaled['alcohol'],x_train_scaled['Malic Acid'])
      ax2.set_title('after normalisation')
```

```
[38]: Text(0.5, 1.0, 'after normalisation')
```



```
[39]: fig , (ax1,ax2) = plt.subplots(ncols = 2,figsize = (12,5))
      ax1.set_title("before Scaling")
      sns.kdeplot(x_train['alcohol'],ax=ax1)
      sns.kdeplot(x_train['Malic Acid'],ax = ax1)
      ax2.set_title("after scaling")
      sns.kdeplot(x_train_scaled['alcohol'],ax=ax2)
      sns.kdeplot(x_train_scaled['Malic Acid'],ax = ax2)
```

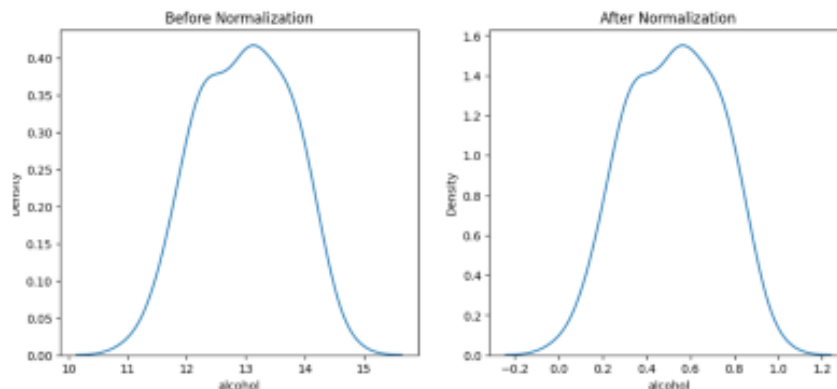
```
[39]: <Axes: title={'center': 'after scaling'}, xlabel='alcohol', ylabel='Density'>
```



- Shape of distributions can change slightly post-normalization.
- Alcohol and Malic Acid now fall between 0 and 1.

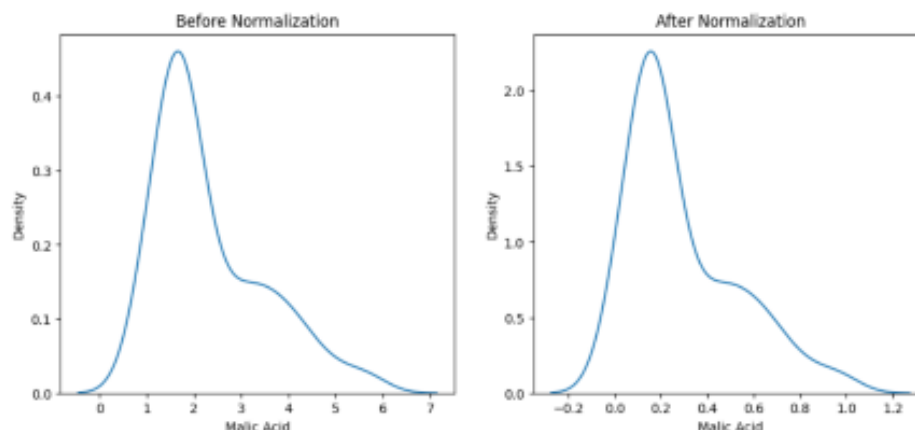
```
[44]: fig, (ax1, ax2) = plt.subplots(ncols = 2, figsize = (12,5))
      ax1.set_title("Before Normalization")
      sns.kdeplot(x_train['alcohol'], ax=ax1)
      ax2.set_title("After Normalization")
      sns.kdeplot(x_train_scaled['alcohol'], ax=ax2)
```

```
[44]: <Axes: title={'center': 'After Normalization'}, xlabel='alcohol', ylabel='Density'>
```



```
[46]: fig, (ax1, ax2) = plt.subplots(ncols=2, figsize = (12,5))
      ax1.set_title("Before Normalization")
      sns.kdeplot(x_train['Malic Acid'], ax=ax1)
      ax2.set_title("After Normalization")
      sns.kdeplot(x_train_scaled['Malic Acid'], ax=ax2)
```

```
[46]: <Axes: title={'center': 'After Normalization'}, xlabel='Malic Acid', ylabel='Density'>
```



- No major difference in scatter plots, but value ranges became comparable.
- Recommended experimenting with other distributions for better understanding.

⚠ Notes on Outliers:

- Normalization does **not** handle outliers well.
- When outliers are present, values get compressed toward 0 (especially in [0–1] range).
- Suggests careful attention to whether distribution shape is preserved.

💡 Additional Insights:

- Recommended comparing before-and-after plots to understand impact.
- Highlighted that not all normalization retains distribution shape.
- In case of heavy outliers, consider alternatives like **Standard Scaling** or **Robust Scaling**.

✅ Best Practices:

- Always visualize data before and after normalization.
- Use `.describe()` to observe statistical changes.
- Choose scaling technique based on data characteristics and model requirements.

2. Mean Normalization

- Formula:

$$x_{\text{scaled}} = \frac{x_i - \text{mean}(X)}{\max(X) - \min(X)}$$

- Explanation:
 - Used to center the data around zero.
 - Values typically range between -1 and 1.
 - If a value is less than the mean, the result is negative; if more, then positive.
- Use Case:
 - When centered data is required.
 - Rarely used; not directly available in Scikit-learn (you must implement it manually).
 - Sometimes replaced by **Standardization** for better results.

3. MaxAbs Scaling

- Formula:

$$x_{\text{scaled}} = \frac{x_i}{\max(|x|)}$$

- Explanation:
 - Scales each value by dividing with the maximum absolute value.
 - Keeps sign of data.
 - Maintains sparsity (i.e., retains zeros in the data).
- Scikit-learn Class:
MaxAbsScaler
- Use Case:
 - Effective when the dataset has many **zero values**.
 - Useful in **sparse datasets**.
 - Rarely used unless working with very specific types of datasets.

4. Robust Scaling

- Formula:

$$x_{\text{scaled}} = \frac{x_i - \text{median}(X)}{\text{IQR}} = \frac{x_i - \text{median}(X)}{Q3 - Q1}$$

- Explanation:
 - Centers data using the median and scales using the interquartile range (IQR).
 - Less sensitive to **outliers**.
- Scikit-learn Class:
RobustScaler
- Use Case:
 - Best when data has **outliers**.
 - Robust to extreme values.
 - Should try this when standard scaling fails due to outliers.

4. Normalization vs. Standardization

First Question to Ask:

"Is **scaling** even required for this feature?"

When to Use Normalization:

- When you **know the min and max values** in advance.
- Example: **Image Processing**
 - Color channel values range from 0 to 255.
 - Use Min-Max scaling to bring values to [0, 1].

When to Use Standardization:

- Most commonly used technique.
- Suitable when feature distribution is unknown or **normal-like**.
- Preferred in algorithms that assume **zero-mean and unit-variance**, such as:
 - Linear regression
 - Logistic regression
 - SVM
 - K-Means

When to Use Robust Scaling:

- When the dataset has **many outliers**.



Practical Tip:

"There's **no universal best scaler** – try multiple ones and experiment."

- Feature scaling is **problem-specific**.
- Try different scalers and **compare performance**.
- Machine learning is all about **experimentation**.